2. `clean.py` expliqué

Nutri-Arbitre — fiche de révision · étape 2 sur 5 du trajet des données (download → **clean** → tools → garde_fou → agent) · **brique 6**

En une phrase : ce fichier lit les données brutes téléchargées (`data/raw_cereales.json`), corrige ou retire ce qui est faux, et écrit un tableau propre (`data/cereales_propres.csv`) avec un **journal** qui explique chaque décision (`data/journal_nettoyage.md`).
Lancer : `python -m nutri.clean`
Pourquoi c'est important : la brique 6 demande de vraies données sales, un journal de nettoyage, et au moins une règle que le fichier ne dit pas, tirée d'un document. Ici : le règlement européen INCO.

La notion clé : le tableau pandas et les « masques »

pandas est une bibliothèque qui manipule des tableaux, comme Excel mais en code. Un tableau s'appelle un **DataFrame** (ici `df`) : une ligne par produit, une colonne par information. Une case vide vaut **NaN** (« Not a Number », valeur manquante).

Presque tout le fichier utilise la même technique en deux temps :

```
hors = (df["sucres"] < 0) | (df["sucres"] > 100)   # 1. masque
df.loc[hors, "sucres"] = None                        # 2. action
```

1. **Le masque** : on pose une question à *toute la colonne d'un coup*. La réponse est une colonne de `True` / `False` , une par produit. `|` veut dire « ou », `&` veut dire « et ».
2. **`df.loc[masque, colonne] = …`** : « dans les lignes où le masque vaut True, et dans cette colonne, mets … ». Mettre `None` vide la case.

produit	sucres	masque hors	après
A	12	False	12
B	150	True	vide
C	-3	True	vide

`masque.sum()` compte les `True` (True vaut 1, False vaut 0) : c'est comme ça qu'on obtient les nombres écrits dans le journal.

Lignes 18 à 35 : les réglages

- `ENTREE` , `SORTIE` , `JOURNAL` : les trois fichiers (lu, écrit, écrit).

- `NUTRIMENTS` : un dictionnaire « nom dans Open Food Facts → notre nom en français ». Par exemple `"sugars_100g"` devient `"sucres"`.
- `EN_GRAMMES` : les colonnes exprimées en grammes pour 100 g (toutes sauf l'énergie, qui est en kcal).
- `LABELS_BIO` : les étiquettes qui veulent dire « bio » (label européen, label AB français). Les accolades `{ }` forment un **ensemble** (*set*) : une liste sans ordre et sans doublons.

Trois petites fonctions d'aide

`normaliser_marque` (lignes 38 à 53)

La même marque est écrite de plusieurs façons. Exemple réel tiré de nos données : `"Kellogg's"`, `"Kellogg's, KELLOGG S PRODUITS ALIMENT SAS"`. Pour pouvoir les regrouper, on les écrit toutes de la même manière :

1. si ce n'est pas du texte, ou si c'est vide → `None` (pas de marque) ;
2. on garde la première marque avant la virgule : `split(",")[0]` ;
3. on enlève les accents (`unicodedata` sépare « é » en « e » + accent, puis on jette l'accent) ;
4. on passe en minuscules et on garde seulement les lettres, chiffres et espaces : l'apostrophe disparaît ;
5. on remplace les espaces multiples par un seul.

Résultat : `"Kellogg's"` → `"kelloggs"` ; `"Nestlé, Nestle France"` → `"nestle"`.

`est_bio` (lignes 56 à 61)

Elle reçoit la liste des labels d'un produit et renvoie `True` dès qu'elle trouve un label bio. Si elle a tout parcouru sans en trouver, elle renvoie `False`.

`aplatir` (lignes 64 à 80)

Les données brutes sont « emboîtées » : chaque produit contient un sous-dictionnaire `nutriments`. Un tableau a besoin d'une seule ligne plate par produit. La fonction parcourt les produits, recopie le code, le nom, la marque, les labels et le Nutri-Score, puis va chercher chaque nutriment dans le sous-dictionnaire. `p.get("code")` renvoie `None` au lieu de planter si l'information manque.

La fonction principale : `nettoyer` (lignes 83 à 172)

Elle applique 10 étapes dans l'ordre. Après chacune, elle ajoute une phrase au `journal` (une liste de textes). On peut les ranger en 5 familles :

A. Retirer des lignes entières (étapes 1, 2, 10)

```
90      avant = len(df)
91      df = df.drop_duplicates(subset="code")
92      journal.append(f"Doublons de code-barres retirés : {avant - len(df)}.")
```

Toujours la même méthode : on compte les lignes *avant*, on retire, et la différence donne le nombre de lignes retirées.

- **1. Doublons** : deux fois le même code-barres = le même produit.
- **2. Sans nom** : `fillna("")` remplace les vides par du texte vide, `str.strip()` enlève les espaces ; on garde les lignes où il reste quelque chose.
- **10. Sans aucune valeur nutritionnelle** : `dropna(how="all")` retire les lignes où *toutes* les colonnes nutritionnelles sont vides. Un produit sans chiffres ne sert à rien.

B. Convertir le texte en nombres (étape 3)

`pd.to_numeric(..., errors="coerce")` transforme `"12.5"` en 12.5. Si c'est impossible (`"traces"`, par exemple), la case devient vide au lieu de planter. On compte les valeurs remplies avant et après pour savoir combien ont été perdues.

C. Vider les valeurs impossibles (étapes 4 et 5)

- **4. Bornes physiques** : pour 100 g de produit, aucun nutriment ne peut peser moins de 0 g ni plus de 100 g. Et l'énergie ne peut pas dépasser 900 kcal pour 100 g, car c'est la valeur de la graisse pure.
- **5. Cohérence** : les sucres font partie des glucides, donc sucres > glucides est impossible. De même, les acides gras saturés font partie des lipides.

Choix important : on vide la *case* fausse, pas toute la ligne. Le produit garde ses autres valeurs, qui sont peut-être justes.

D. Les règles du règlement INCO, absentes du fichier (étapes 6 et 7)

Le règlement européen n° 1169/2011 (« INCO », annexe XIV) fixe comment calculer les valeurs affichées sur les emballages. Le fichier ne le dit nulle part : c'est la « règle obtenue dans un document » que demande la brique 6.

```
127     a_completer = df["sel"].isna() & df["sodium"].notna()
128     df.loc[a_completer, "sel"] = df.loc[a_completer, "sodium"] * 2.5
```

- **6. Sel = sodium × 2,5** : si le sel manque mais que le sodium est connu, on calcule le sel. Le masque se lit : « sel vide **et** sodium rempli ».
- **7. Énergie = 4 × glucides + 4 × protéines + 9 × lipides + 2 × fibres** (en kcal). La règle sert deux fois :
  - pour **compléter** : si l'énergie manque et que les trois macronutriments sont connus, on la calcule (arrondie à l'unité) ;
  - pour **contrôler** : si l'énergie déclarée s'écarte de plus de 20 % du calcul, les valeurs de la fiche se contredisent. `calcul.where(calcul > 0)` évite une division par zéro.

```
    ecart = (df["energie_kcal"] - calcul).abs() / calcul.where(calcul > 0)
    suspect = (ecart > 0.20).fillna(False)
    avant = len(df)
    df = df[~suspect]          # ~ = « non » : on garde les non suspects
```

Pourquoi retirer la ligne entière ici, alors qu'ailleurs on vide seulement la case ? Quand une valeur est impossible (150 g de sel), on sait *laquelle* est fausse : on la vide et on garde le reste. Mais quand l'énergie ne colle pas avec les macronutriments, **on ne peut pas savoir laquelle est fausse** : l'énergie, les glucides, les lipides ? Les données ont été mal remplies ou remplies en partie, donc toute la fiche est douteuse. Exemple réel : « Muesli croustillant chocolat » déclare 437 kcal, ce qui est normal pour un muesli, mais ses macronutriments ne donnent que 218 kcal. Ses 33 g de glucides sont probablement faux, et donc peut-être ses sucres aussi. Ces fiches ne coûtent que 4 produits sur 957 : on les retire. *(Décision prise le 1er novembre 2026.)*

E. Mettre les libellés au propre (étapes 8 et 9)

- **8. Nutri-Score** : on passe en minuscules, puis tout ce qui n'est pas a, b, c, d ou e (« unknown », « not-applicable ») devient vide. Le `~` veut dire « non » : `~df["nutriscore"].isin(...)` = « n'est pas dans la liste ».
- **9. Marques et bio** : `.map(fonction)` applique une fonction à chaque case d'une colonne. On l'utilise avec `normaliser_marque` et `est_bio`.

À la fin (lignes 170 à 172), on écrit le nombre de produits à l'arrivée, on ne garde que les colonnes utiles dans un ordre fixe, et `reset_index(drop=True)` renumérote les lignes de 0 à n−1.

`main` (lignes 175 à 182)

Elle lit le JSON brut, appelle `nettoyer`, écrit le tableau en CSV (`index=False` : sans la colonne des numéros de ligne) et le journal en Markdown, avec un tiret devant chaque phrase.

Ce qui s'est vraiment passé sur nos données (téléchargées le 30 octobre 2026) :

Étape	Effet
Départ	1 000 produits
Doublons	0
Sans nom	28 retirés
Énergie impossible	1 valeur vidée
Sucres > glucides, AGS > lipides	0
Sel calculé depuis le sodium	0 produit
Énergie calculée (INCO)	1 produit
Énergie incohérente (fiche douteuse)	4 retirés
Nutri-Score invalide	30 vidés (unknown, not-applicable)
Marques	294 libellés bruts → 177 marques
Sans aucune valeur	15 retirés
Arrivée	**953 produits** (543 bio, 410 non bio)

À savoir dire honnêtement :

- La règle du sel n'a servi pour aucun produit : Open Food Facts calcule déjà le sel lui-même. On la garde quand même, car elle protège contre de futures données où le sel manquerait. La règle de l'énergie, elle, a servi deux fois : 1 valeur complétée et 4 fiches incohérentes repérées et retirées.
- La normalisation des marques a une limite : « KELLOGG S PRODUITS ALIMENT SAS » ne devient pas « kelloggs ». Un même fabricant peut donc apparaître sous deux noms.

Questions d'examinateur

1. Que fait la ligne `df.loc[hors, col] = None` ?
2. Pourquoi vide-t-on la case fausse au lieu de retirer tout le produit ?
3. Quelle est la « règle absente du fichier » demandée par la brique 6, et d'où vient-elle ?
4. Pourquoi est-il impossible d'avoir plus de sucres que de glucides ?
5. Comment le journal obtient-il le nombre de lignes retirées ?
6. Que se passe-t-il si on appelle `nettoyer([])`, avec une liste vide ?
7. Pourquoi retire-t-on toute la fiche quand l'énergie ne colle pas, alors qu'ailleurs on vide seulement la case ?
8. La règle du sel n'a servi pour aucun produit. Est-ce qu'elle est inutile ?

Essaie d'y répondre avec tes mots avant de lire le corrigé à la page suivante.

Corrigé

1. `df.loc[hors, col] = None` : dans toutes les lignes où le masque `hors` vaut True, et seulement dans la colonne `col`, on vide la case. Les autres lignes et les autres colonnes ne changent pas.
2. **Vider la case plutôt que la ligne** : une seule valeur est fausse, les autres sont peut-être justes. Si un produit a un sel impossible (150 g), son sucre et ses lipides restent utilisables. Retirer la ligne ferait perdre des informations pour rien.
3. **La règle absente du fichier** : le règlement européen n° 1169/2011, dit « INCO », annexe XIV. Il donne sel = sodium × 2,5 et énergie = 4 × glucides + 4 × protéines + 9 × lipides + 2 × fibres. Le fichier Open Food Facts ne contient pas ces règles : on les a prises dans le texte du règlement.
4. **Sucres et glucides** : les sucres sont une *partie* des glucides (les glucides = sucres + amidon…). Une partie ne peut pas être plus grande que le tout : si c'est le cas, une des deux valeurs est une erreur de saisie.
5. **Le nombre de lignes retirées** : on enregistre `avant = len(df)`, on retire les lignes, puis on écrit `avant - len(df)`. Pour les cases vidées, on utilise `masque.sum()`, qui compte les True.
6. **Liste vide** : `aplatir` crée un tableau vide mais avec les bonnes colonnes. Chaque étape trouve 0 ligne à traiter, et le journal finit par « Arrivée : 0 produits. ». Rien ne plante, et c'est vérifié par le test `test_liste_vide`.
7. **La règle du sel** : elle n'a rien changé *sur ces données-là*, parce qu'Open Food Facts fournit déjà le sel. Mais si on retélécharge un jour des produits où seul le sodium est indiqué, elle complétera le sel. Et le test `test_regle_inco_sel_depuis_sodium` prouve qu'elle fonctionne. Le dire honnêtement montre qu'on a regardé le journal.
8. **Toute la fiche plutôt que la case** : quand une valeur est impossible, on sait laquelle est fausse, donc on vide seulement celle-là. Quand l'énergie déclarée et la formule INCO ne collent pas, deux valeurs se contredisent et on ne peut pas savoir laquelle est fausse : la fiche a été mal remplie ou remplie en partie, et toute la ligne devient douteuse. Comme ça ne concerne que 4 produits sur 957, les retirer ne fait presque rien perdre et rend les calculs plus sûrs.